

ClassLedger

Registre pédagogique sur blockchain privée

Auteur	Cheikhna Ibrahim DIAGANA
Numéro étudiant	63215
Établissement	ISEP
Promotion	2023-2026
Enseignant responsable	Nouredine Tamani
Plateforme	Hyperledger Besu sur Microsoft Azure
Application	Python / FastAPI
Contrat	Solidity (EVM)
Dernière validation	9 septembre 2026 — contrat V4
Révision documentaire	9 septembre 2026

Sommaire

Résumé

1. Problème à résoudre

2. Étude des solutions et choix de plateforme

3. Architecture retenue

4. Données du contrat et règles d'accès

5. Réseau et consensus

6. Application et sécurité

7. Démarche de réalisation

8. Validation

9. Déploiement Azure

10. Limites du projet

Conclusion

Références

Résumé

ClassLedger est une application de partage des ressources d'une classe sur une blockchain privée. La description et l'organisation sont publiques ; les cours et travaux pratiques sont réservés aux inscrits ; les examens et leurs corrections ne sont ouverts que vingt-quatre heures après l'épreuve ; enfin, chaque étudiant ne peut consulter que sa propre note.

L'objectif est de faire appliquer ces règles par la blockchain, et non par la seule interface web. La solution utilise un smart contract Solidity, c'est-à-dire un programme exécuté sur la chaîne, avec quatre nœuds Hyperledger Besu. Ces nœuds valident les blocs avec le protocole QBFT : trois validateurs sur quatre doivent participer pour continuer à les finaliser. L'application FastAPI relie le navigateur à ce réseau. Les fichiers restent sur disque ; le contrat conserve leurs métadonnées et leur empreinte SHA-256, qui permet de vérifier qu'ils n'ont pas changé.

La réalisation a progressé par étapes, des règles du contrat au déploiement sur Azure. La validation comprend 24 tests automatisés, un scénario complet dans l'application et des arrêts de validateurs. Ces essais ont permis de vérifier la copie des blocs, le blocage lorsque trop de nœuds sont arrêtés, puis leur rattrapage au redémarrage. Un fichier modifié sur disque est également détecté. La démonstration Azure utilise HTTPS, sans exposer les ports de la blockchain.

1. Problème à résoudre

Le sujet définit quatre règles d'accès et plusieurs propriétés de la blockchain à démontrer. Le tableau suivant associe chaque exigence à une vérification concrète.

Réf.	Exigence	Critère observable
R1	Description et organisation publiques	lecture possible sans compte
R2	Cours et TP réservés aux inscrits	refus avant inscription, accès après approbation
R3	Examens et corrections ouverts après 24 h	refus avant l'échéance, accès ensuite
R4	Note personnelle uniquement	un étudiant ne lit jamais la note d'un autre
B1	Réplication et auditabilité	les quatre nœuds possèdent le même bloc ; les informations publiques sont lisibles
B2	Immutabilité des publications	documents, informations de classe et notes sont conservés ; une révision ajoute une entrée
B3	Distribution P2P	chaque validateur possède trois pairs
B4	Consensus	3/4 finalisent ; 2/4 ne finalisent pas

Les règles d'accès sont définies dans le contrat, indépendamment des écrans. Pour une lecture privée, l'application transmet l'adresse de l'utilisateur au contrat, qui la reçoit comme `msg.sender` et vérifie ses droits.

Cette lecture `eth_call` n'est toutefois pas signée : un client disposant du RPC peut choisir `from`. La passerelle fixe cette adresse depuis la session authentifiée. Le contrôle de lecture repose donc aussi sur le RPC privé et, pour les notes, sur le chiffrement ; `msg.sender` seul n'authentifie pas un appel de lecture.

Le projet se limite à une classe, un enseignant et des étudiants fictifs. La connexion institutionnelle, la messagerie et la gestion de plusieurs classes restent hors périmètre. L'objectif est d'étudier le fonctionnement de la blockchain, et non de construire un environnement numérique de travail complet.

2. Étude des solutions et choix de plateforme

Le choix de Besu repose sur une comparaison des offres cloud citées dans le sujet et de deux plateformes open source adaptées aux réseaux privés.

2.1 Fournisseurs cloud

Microsoft Azure. Azure Blockchain Service a été retiré en septembre 2021 ; la documentation de migration proposait notamment un déploiement autogéré sur des machines virtuelles [2]. Azure propose encore Confidential Ledger, un registre append-only protégé par des enclaves matérielles, accessible par API REST et capable de fournir des reçus de transactions. Ce service convient à un journal d'audit, mais ne fournit pas l'environnement EVM/Solidity retenu pour exécuter les règles R2 à R4 [1].

Amazon Web Services. Amazon Managed Blockchain fournit des accès gérés à plusieurs réseaux publics ainsi qu'une offre Hyperledger Fabric pour les réseaux privés. La facturation dépend notamment des nœuds, du stockage, des requêtes et du transfert. Cette solution réduit l'administration, mais donne moins de contrôle direct sur le réseau. L'auto-hébergement répond mieux au besoin d'arrêter et d'inspecter chaque validateur, sans ajouter le coût d'un service géré [3].

IBM et Oracle. IBM indique que Blockchain Platform 2.5.4 n'est plus publié et oriente vers le support Hyperledger Fabric [4]. Oracle maintient une offre blockchain managée, riche en intégrations et destinée aux réseaux d'entreprise. Ces intégrations ne sont pas nécessaires pour le périmètre mono-classe retenu [5].

2.2 Plateformes open source

Hyperledger Fabric offre des identités, des canaux, des politiques d'endorsement et des données privées. Ces mécanismes sont pertinents dans un consortium, mais exigent davantage de composants : autorités de certification, peers, service d'ordre et configuration des canaux. Cette option n'a pas été retenue afin de limiter la complexité de configuration du projet [8].

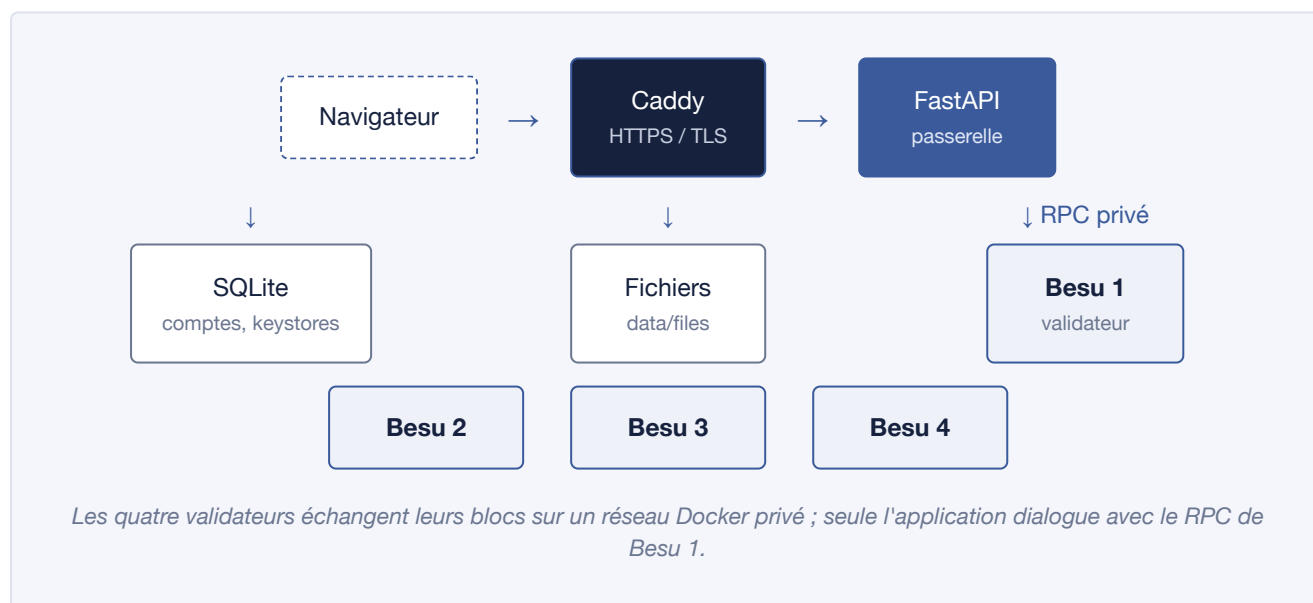
Hyperledger Besu est un client Ethereum open source. Il exécute des contrats Solidity, expose les API compatibles Ethereum et prend en charge QBFT pour les réseaux privés. La documentation Besu recommande QBFT pour ce contexte et précise qu'il faut quatre validateurs pour obtenir une tolé-

rance byzantine [6]. Son modèle EVM s'intègre directement à `web3.py`.

Critère	Azure Confidential Ledger	AWS / Fabric géré	Fabric auto-hébergé	Besu auto-hébergé
Contrat Solidity	Non	Non pour Fabric	Non	Oui
Consensus visible et testable	Peu	Partiellement	Oui	Oui
Coût logiciel	Service payant	À l'usage	Gratuit	Gratuit
Complexité pour une classe	Faible, mais inadapté	Élevée	Élevée	Modérée
Intégration Python/EVM	Non	Indirecte	SDK Fabric	Directe

La solution retenue est **Besu auto-hébergé sur Azure**. Besu fournit la blockchain ; Azure héberge uniquement la machine de démonstration. Le projet n'utilise donc pas l'ancien Azure Blockchain Service. La VM utilise un **abonnement Azure personnel**. Elle est désallouée après les démonstrations pour arrêter les frais de calcul ; le disque et l'IP restent toutefois facturables. Le projet peut aussi être installé et exécuté entièrement en local.

3. Architecture retenue



Docker Compose regroupe les cinq services principaux : quatre nœuds Besu et l'application FastAPI. Sur Azure, Caddy gère la connexion HTTPS. Aucun port RPC ou P2P n'est publié sur l'hôte ; à l'intérieur du réseau Docker privé, l'application dialogue avec le RPC de `validator1`, et les scripts de vérification interrogent les quatre nœuds pour comparer leurs copies.

Les données sont réparties entre deux emplacements :

- **sur la chaîne** : description, organisation, inscriptions, métadonnées, empreintes de fichiers, liens de versions et notes chiffrées ;
- **hors chaîne** : contenu des fichiers, comptes web, correspondance entre noms et adresses, keystores chiffrés et secrets applicatifs.

Les fichiers pédagogiques restent hors chaîne pour éviter de copier plusieurs mégaoctets sur chaque nœud. L'empreinte SHA-256 ne prend que 32 octets. Au téléchargement, FastAPI recalcule cette empreinte et la compare à celle du contrat. Si elles diffèrent, il refuse le fichier. Cela permet de vérifier son intégrité sans stocker tout son contenu dans les blocs.

4. Données du contrat et règles d'accès

4.1 Inscription

Le contrat distingue quatre états : `NONE` , `PENDING` , `ENROLLED` et `REJECTED` . Un compte étudiant demande son inscription pour lui-même. Seule l'adresse de l'enseignant peut ensuite approuver ou refuser cette demande. Un refus n'efface pas l'historique et autorise une nouvelle demande. Une seconde voie d'écriture, `enroll` , permet à l'enseignant d'inscrire directement une adresse connue : elle sert à initialiser la liste de classe, par exemple lors du chargement des comptes de démonstration. Les deux chemins restent tracés par des événements distincts.

Dans l'application, chaque étudiant possède une adresse Ethereum. La transaction de demande est signée avec la clé associée à cette adresse. Cette clé est toutefois conservée par la passerelle dans un keystore, un fichier contenant la clé sous forme chiffrée. La signature est celle du compte étudiant, mais c'est le serveur qui détient sa clé et signe pour lui. C'est ce qu'on appelle un modèle custodial.

La création préalable du compte web et du keystore dans SQLite est hors chaîne ; elle ne constitue pas encore une demande d'inscription au contrat.

4.2 Documents

Chaque document est une structure `Material` comportant un type, un titre, une référence de fichier, son SHA-256, une date d'examen éventuelle, un horodatage et un lien vers une version précédente.

La fonction interne de visibilité applique une règle simple :

1. l'enseignant voit tous les documents ;
2. un non-inscrit ne voit aucun contenu privé ;
3. cours et TP sont ouverts aux inscrits ;
4. examens et corrections exigent en plus `block.timestamp >= examAt + 24 hours` .

Une correction doit référencer un examen ou une correction déjà rattachée à cet examen, et hériter de sa date. Le verrou de vingt-quatre heures reste ainsi celui de l'épreuve, même après plusieurs versions.

Le verrou porte sur le contenu : un inscrit peut consulter les métadonnées d'un examen encore verrouillé. Les dates sont converties depuis `Europe/Paris` par défaut ; le contrat applique un délai de 86 400 secondes.

Aucune fonction ne modifie ni ne supprime un document publié. Le contrat V4 applique aussi cette conservation aux informations de classe et aux notes : une publication ajoute une version, sans remplacer les précédentes. `classInfoAt` permet de relire les informations publiques à un indice donné ; les lectures habituelles renvoient la dernière version. En revanche, le statut d'inscription peut changer ; chaque changement est enregistré par un événement.

4.3 Notes

Le contrat associe une liste de versions à l'adresse de l'étudiant. `myGrade()` renvoie sa dernière note ; `gradeOf()` limite la lecture à l'étudiant concerné ou à l'enseignant. `gradeAt` et `gradeVersionCount` protègent l'historique avec la même règle. L'interface affiche la dernière note ; les versions antérieures restent consultables par les appels autorisés du contrat. Aucun événement ne contient la valeur de la note.

Une limite importante est qu'un administrateur de nœud peut inspecter le stockage brut de la chaîne sans passer par ces fonctions. Les notes sont donc chiffrées avec **AES-256-GCM** avant leur envoi dans une transaction. L'adresse de l'étudiant est utilisée comme donnée authentifiée : déplacer le texte chiffré vers une autre adresse fait échouer sa vérification. Les validateurs répliquent la valeur chiffrée produite par l'application, pas la note en clair. Le contrat stocke une chaîne sans imposer lui-même le format chiffré ; le lecteur conserve une compatibilité avec d'anciennes valeurs non chiffrées.

5. Réseau et consensus

Le réseau utilise Besu 26.2.0, une chaîne privée d'identifiant `1337` et quatre validateurs statiques. Le genesis fixe une période de bloc de 2 secondes et un délai de tour de 4 secondes. Le prix minimal du gas est nul : aucune monnaie réelle n'est nécessaire pour signer les transactions du prototype.

QBFT est un protocole byzantin entre validateurs autorisés, utilisé ici dans un réseau Proof of Authority. Avec quatre nœuds, trois signatures sont nécessaires pour finaliser un bloc, sous les hypothèses de sécurité du protocole. Le nombre maximal de validateurs défaillants tolérés, noté f , dépend du nombre de validateurs n :

$$f = \left\lfloor \frac{n-1}{3} \right\rfloor$$

Pour $n = 4$, le réseau tolère donc une défaillance. Si deux validateurs sont arrêtés, il ne finalise plus de nouveau bloc. Il s'arrête plutôt que de valider avec un nombre insuffisant de participants. Ce comportement a été observé lors des essais, conformément au seuil décrit dans la documentation Besu.

Les quatre nœuds s'appuient sur le même genesis, mais possèdent chacun leur clé et leur volume de chaîne. Ils échangent les blocs sur un réseau Docker privé. Cette configuration permet de tester la réplique et le consensus. Elle ne protège pas contre une panne de la machine qui héberge tous les conteneurs.

6. Application et sécurité

FastAPI et Jinja2 produisent les pages HTML côté serveur [9]. Cela évite d'ajouter une application frontend séparée. La passerelle assure l'authentification web, la signature des transactions, le dépôt des fichiers et la traduction des refus du contrat en messages compréhensibles [10].

Les protections mises en place sont les suivantes :

- mots de passe hachés avec Argon2 ;
- clés étudiantes dans des keystores chiffrés, avec dérivation de clé par scrypt ;
- jeton CSRF lié à la session pour chaque opération d'écriture ;
- limitation des tentatives de connexion et d'inscription ;
- cookie `HttpOnly`, `SameSite=Strict`, durée de deux heures, `Secure` sur Azure ;
- contrôle de l'en-tête `Host` et en-têtes CSP, HSTS, `nosniff` et anti-frame ;
- taille des fichiers limitée à 10 Mio, noms assainis et permissions `0600` ;
- validation obligatoire de secrets indépendants au démarrage en production ;
- documentation OpenAPI désactivée en production.

Les secrets ne sont pas intégrés à l'image. Ils sont chargés depuis `.env`, exclu de Git. Le script de déploiement génère des valeurs aléatoires indépendantes pour les mots de passe, les sessions, les keystores et le chiffrement des notes. Les clés EVM des trois comptes préchargés constituent une exception volontaire : ce sont des clés de test publiques, versionnées dans `.env.example`, utilisables uniquement sur cette chaîne isolée sans valeur. Une exploitation réelle devrait les remplacer.

7. Démarche de réalisation

La réalisation distingue les tests des règles d'accès et ceux du réseau Besu. Les premiers s'exécutent dans une EVM en mémoire ; les seconds utilisent les quatre validateurs Docker. Cette séparation permet, par exemple, de vérifier le délai d'ouverture d'un examen sans attendre vingt-quatre heures, puis de tester la validation des transactions avec de véritables nœuds Besu.

1. **Règles.** Les conditions d'accès sont définies indépendamment des pages web.
2. **Contrat.** Les premiers tests utilisent une EVM en mémoire, un environnement qui exécute le bytecode Solidity, avant le déploiement sur Besu.
3. **Réseau.** Un script crée le genesis et les quatre identités QBFT. Les contrôles portent ensuite sur les volumes séparés et les connexions entre nœuds.
4. **Application.** Les pages web sont reliées aux appels du contrat. Les lectures utilisent l'adresse de l'utilisateur connecté.
5. **Inscription.** La demande par l'étudiant et l'approbation par l'enseignant complètent l'inscription directe d'une adresse par le professeur.
6. **Sécurité.** Les protections web, le chiffrement des notes et HTTPS sont intégrés avant les essais sur Azure.

7. **Commandes.** `make demo` regroupe l'installation ; `make verify` lance les tests, le parcours HTTP et l'essai de consensus.

Le tableau suivant présente les principaux problèmes rencontrés et leurs corrections :

Observation	Diagnostic	Correction
déploiement Solidity sans code	bytecode compilé avec <code>PUSH0</code> , absent du palier Berlin	compilation explicite pour <code>evm_version="berlin"</code>
erreur après évolution du contrat sur Azure	ancienne adresse réutilisée malgré une ABI incompatible	vérification du code présent et de <code>CONTRACT_VERSION</code> , sinon redéploiement
CSS bloqué en HTTPS	URL générée en HTTP derrière Caddy	prise en compte de <code>X-Forwarded-Proto</code> ; port API publié en loopback sur l'hôte, écoute interne au conteneur sur <code>0.0.0.0</code>
suite du script SSH ignorée	<code>docker compose exec</code> consommait l'entrée standard	exécution avec <code>-T --interactive=false</code>
clés réseau impossibles à nettoyer	fichiers créés par Docker avec le propriétaire root	génération avec l'UID/GID de l'hôte

Le problème de `PUSH0` a notamment montré qu'un contrat compilé sans erreur ne garantit pas sa compatibilité avec le réseau cible. La compilation explicite pour Berlin corrige ce décalage. Sur Azure, le contrôle de `CONTRACT_VERSION` répond à un autre problème : une adresse contenant du code peut encore désigner une ancienne version du contrat. Ces vérifications sont intégrées aux scripts pour ne pas dépendre de corrections manuelles à chaque déploiement.

8. Validation

8.1 Tests automatisés

La suite `pytest` comprend **24 tests**. Ils vérifient les informations publiques, cours, TP, examen, correction, notes, rôles, inscription, refus, réinscription et conservation des anciennes versions. La suite couvre aussi le rattachement obligatoire d'une correction à son examen et l'héritage de sa date, le chiffrement des notes, la vérification du texte chiffré, CSRF, les secrets de production et l'isolation des chemins de fichiers. Deux tests V4 vérifient la conservation des anciennes informations de classe et notes, ainsi que l'accès privé à ces dernières.

Un dernier défaut concernait le tableau enseignant : il pouvait afficher un examen comme ouvert aux étudiants en utilisant les droits du professeur. La colonne « Ouverture aux inscrits » remplace ce badge et indique la date d'ouverture des examens et corrections, ou l'accès dès inscription pour les cours et TP. Un test vérifie ce rendu pour éviter que le défaut revienne.

8.2 Parcours de bout en bout

Le script `smoke.sh` vérifie un parcours complet par HTTP sur le réseau Besu. Il publie un cours, un examen encore verrouillé et une note depuis le compte enseignant. Le premier étudiant doit pouvoir télécharger le cours et lire sa note, mais pas télécharger l'examen. Le second ne doit pas voir la note du premier. Le script crée aussi un compte et vérifie son passage de la demande d'inscription à l'approbation. Enfin, il modifie volontairement un fichier sur disque : le téléchargement est refusé parce que son SHA-256 ne correspond plus à celui du contrat. Ce dernier essai vérifie le contrôle du fichier lui-même, pas seulement la présence d'une empreinte dans la blockchain.

8.3 Réplication et quorum

Pour vérifier le réseau, le script `compliance.sh` interroge le RPC de chacun des quatre nœuds. Voici les résultats obtenus sur Azure le 9 septembre 2026 avec le contrat V4. Les mêmes essais ont aussi réussi en local :

Essai	Résultat observé
état initial	4 validateurs, 3 pairs chacun et même hash au bloc comparé
arrêt d'un validateur	transaction finalisée par les 3 validateurs restants
arrêt d'un second validateur	transaction toujours en attente après le délai
restauration	rattrapage, retour à 3 pairs et même hash sur les quatre nœuds

Ces essais vérifient que les nœuds possèdent les mêmes blocs, que le seuil de trois validateurs est respecté et que les nœuds redémarrés rattrapent les autres.

La validation comprend aussi une recompilation du contrat avec la version prévue du compilateur. Le fichier produit contient l'ABI, qui décrit les fonctions accessibles, et le bytecode exécuté par l'EVM. Son SHA-256 est resté identique après recompilation : le fichier présent dans le dépôt correspond donc au résultat de cette compilation.

9. Déploiement Azure

La démonstration est hébergée sur une VM Ubuntu 22.04 de type `Standard_B2ms`, disposant de 2 vCPU, de 8 Gio de mémoire et d'un disque Standard SSD de 30 Gio, dans la région France Central. Cette machine regroupe les quatre validateurs Besu et l'application FastAPI. Docker est installé par `cloud-init`, puis les scripts de déploiement transfèrent le code avec `rsync`, sans copier les secrets ni les données de l'environnement local.

L'accès à l'application passe par Caddy, qui gère HTTPS avec un certificat Let's Encrypt. Seuls les ports web 80 et 443 sont accessibles publiquement ; SSH est limité à l'adresse de l'administrateur. Les ports RPC et P2P de Besu ne sont pas exposés. Les utilisateurs accèdent ainsi à ClassLedger par le navigateur, sans connexion directe aux nœuds blockchain.

Les tests réalisés sur Azure ont confirmé les résultats obtenus en local. Un redémarrage de la VM a également permis de vérifier la reprise automatique des services, configurés avec `restart: unless-stopped`. Besu demande toutefois un temps de démarrage supplémentaire : la disponibilité de la page web ne suffit pas à confirmer celle de la chaîne.

Lors d'une mise à jour, le script réutilise le contrat existant s'il est compatible. Sinon, il déploie une nouvelle instance, sans migrer automatiquement les anciennes données. L'ancien contrat reste présent sur la chaîne, mais l'application utilise la nouvelle adresse.

L'hébergement repose sur un abonnement Azure personnel. Le tarif de calcul relevé pour cette configuration est de 0,0944 USD/h, hors disque et adresse IP [7]. Pour limiter les dépenses, la VM est désallouée après les démonstrations et un arrêt automatique est configuré à 23 h UTC. Un budget mensuel de 10 EUR, assorti d'une alerte à 80 %, permet de suivre les dépenses ; il ne plafonne pas leur montant. Le disque et l'adresse IP restent facturables lorsque la VM est désallouée.

10. Limites du projet

La première limite est le choix d'une **seule VM**. Les quatre validateurs ont bien leurs propres clés et leurs propres volumes, mais ils dépendent de la même machine. Une panne de cette VM interrompt donc l'ensemble du service. Pour un usage réel, il faudrait répartir les validateurs entre plusieurs machines et plusieurs organisations.

Les copies sont détenues par les validateurs, pas par chaque étudiant. De plus, la perte de `validator1` peut laisser le consensus progresser à trois mais prive la passerelle de son point d'accès RPC unique. Le réseau privé et les pairs statiques ne constituent pas, à eux seuls, un permissionnement Besu complet.

La deuxième limite est la **confiance dans le serveur**. QBFT fait participer plusieurs validateurs, mais ils sont tous gérés par le même administrateur dans cette démonstration. L'application conserve aussi les comptes, les fichiers et les clés étudiantes. L'enseignant garde le droit d'approuver les inscriptions et de publier les contenus. Utiliser une blockchain ne supprime donc pas les responsabilités de l'administrateur et de l'enseignant.

Enfin, le chiffrement protège les notes contre la lecture directe du stockage de la chaîne, mais le serveur détient la clé de déchiffrement. Les fichiers ne sont pas répliqués entre validateurs : leur empreinte permet de contrôler leur intégrité, mais pas de les récupérer s'ils sont perdus. Un usage réel demanderait des portefeuilles côté client, une gestion institutionnelle des identités, des validateurs répartis et un stockage de fichiers sauvegardé ou distribué.

L'hébergement en France ne garantit pas la conformité RGPD. Les identités, adresses et historiques demanderaient une étude spécifique avant un usage réel. Les essais de quorum portent sur des arrêts de nœuds, pas sur toutes les attaques byzantines ni sur la capacité en charge du système.

Conclusion

ClassLedger met en œuvre les fonctions demandées pour une classe : publier des ressources, gérer les inscriptions, appliquer le délai de vingt-quatre heures et limiter la consultation des notes. Le contrat conserve les anciennes versions, et les quatre validateurs maintiennent une copie du registre. Les fichiers restent hors chaîne, avec une empreinte qui permet de détecter une modification.

Les essais du 9 septembre 2026 précisent ce que cette réalisation permet de démontrer : une transaction est finalisée avec trois validateurs, reste en attente avec deux, puis les copies convergent après restauration. Le refus d'un fichier altéré vérifie également le lien entre le document conservé sur disque et son empreinte enregistrée dans le contrat.

La principale limite tient au choix d'héberger les quatre validateurs et l'application sur une seule VM Azure. Cette configuration permet de présenter le consensus et les règles d'accès, mais pas de résister à la perte de cette VM. De même, le chiffrement des notes ne retire pas au serveur la responsabilité de protéger les clés. ClassLedger répond ainsi au périmètre pédagogique retenu ; son utilisation avec de véritables données étudiantes demanderait d'abord de revoir l'hébergement, la gestion des identités et la sauvegarde des fichiers.

Références

Les sources officielles ci-dessous correspondent aux passages cités dans le rapport.

1. Microsoft Learn, [About Azure confidential ledger](#).
2. Microsoft, [Migrate Azure Blockchain Service](#), archive officielle : retrait du service le 10 septembre 2021 et alternatives autogérées sur VM.
3. Amazon Web Services, [Amazon Managed Blockchain pricing](#) : offres publiques, réseaux privés Hyperledger Fabric et postes de facturation.
4. IBM Documentation, [IBM Blockchain Platform 2.5.4 — product version no longer published](#).
5. Oracle, [Oracle Blockchain](#).
6. Hyperledger Besu, [Configure QBFT consensus](#).
7. Microsoft Azure, [Retail Prices API — requête Standard B2ms](#), [France Central](#) et [documentation de l'API](#). Tarif relevé le 5 septembre 2026 : `Virtual Machines BS Series`, compteur `B2ms`, 0,0944 USD/h, hors Windows ; la réponse de l'API évolue avec les tarifs.
8. Hyperledger Fabric, [Introduction](#).
9. FastAPI, [Templates](#) : rendu HTML avec Jinja2 et intégration des fichiers statiques.
10. web3.py, [Contracts](#) : ABI, déploiement, lectures `call`, transactions `build_transaction` et tests sur EVM en mémoire.